



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 17

“Matriz de led 7x5”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 13 de marzo de 2026

INTRODUCCIÓN

Matriz de led 7x5

Una matriz de LEDs de 7x5 es un dispositivo de visualización compuesto por 35 diodos emisores de luz organizados en 7 filas y 5 columnas. Es una de las configuraciones más populares en electrónica para mostrar caracteres alfanuméricos, símbolos y gráficos pequeños de forma clara y eficiente.

Configuración Interna (Ánodo y Cátodo)

Para ahorrar pines, los LEDs no se conectan de forma individual (lo que requeriría 35 pines más tierra). En su lugar, se comparten conexiones:

- **Matriz de Ánodo Común en Filas:** Todos los ánodos de los LEDs en una misma fila están conectados entre sí.
- **Matriz de Cátodo Común en Columnas:** Todos los cátodos de los LEDs en una misma columna están conectados entre sí.

(También existe la configuración inversa, dependiendo del fabricante).

El Método de Multiplexación

Debido a esta estructura compartida, no puedes encender todos los LEDs que quieras simultáneamente de forma estática. Se utiliza la multiplexación por barrido:

1. **Activación de Columna:** Se energiza la primera columna (C1).
2. **Envío de Datos:** Se envían los estados (encendido/apagado) de los 7 LEDs de esa columna a través de las filas (R1-R7).
3. **Persistencia de la Visión (POV):** Se apaga la C1 y se repite el proceso con la C2, C3, C4 y C5 a una velocidad superior a **60 Hz**.
4. **Resultado:** El ojo humano no percibe el parpadeo y ve una imagen estable y completa.

Hardware de Control

Controlar 12 pines (7 filas + 5 columnas) puede agotar rápidamente los puertos de un microcontrolador. Por ello, es común usar:

- **Registros de Desplazamiento:** Permiten controlar las filas usando solo 3 pines del microcontrolador.
- **Decodificadores:** Para seleccionar de forma rápida qué columna activar.
- **Drivers de Corriente:** Dado que una fila puede tener 5 LEDs encendidos a la vez, el microcontrolador podría no soportar la corriente necesaria; estos chips actúan como amplificadores.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 17, el circuito a armar es el siguiente:

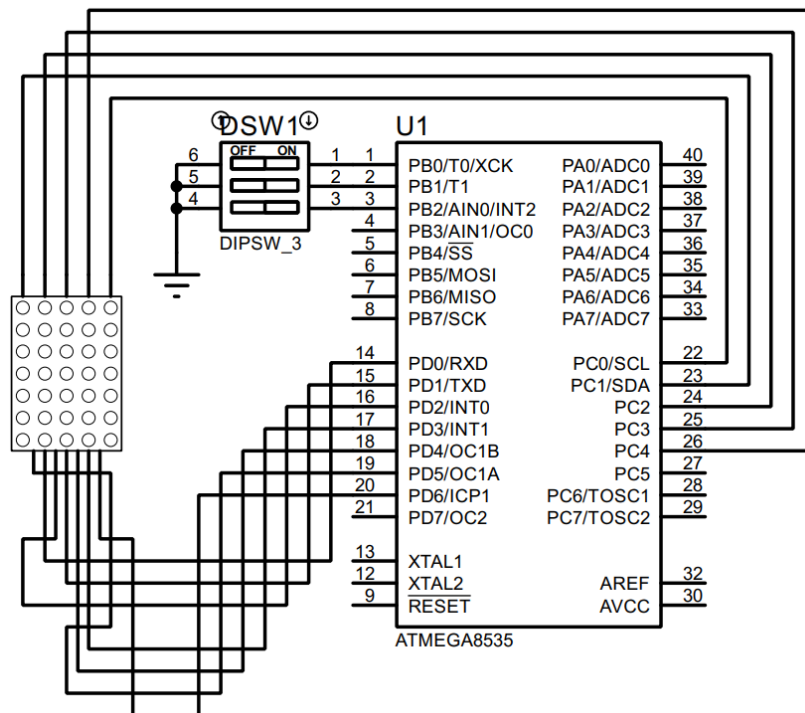


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen, el registro D se configura como de salida (PD0-PD6) al igual que el registro C (PC0-PC4), mientras que el puerto B como de entrada (PB0-PB2) activando las resistencias de pull-up.

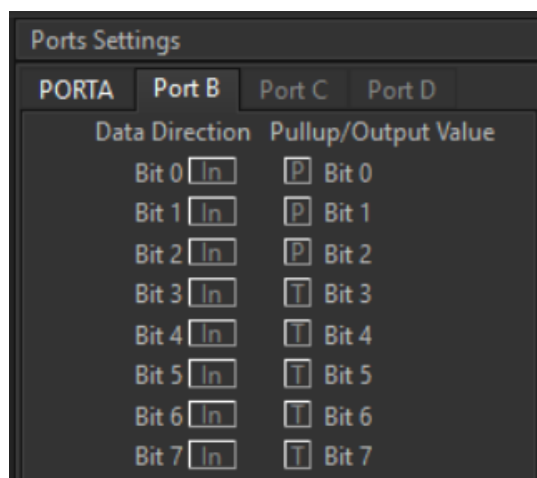


Imagen 2. Configuración del puerto B.

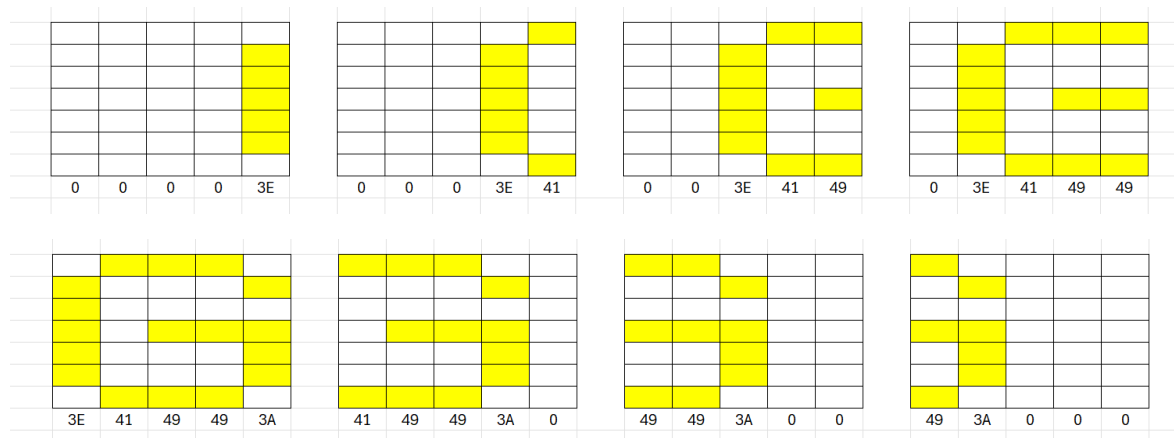
Ports Settings			
PORTA	Port B	Port C	Port D
Data Direction		Pullup/Output Value	
Bit 0	Out	0	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	In	T	Bit 5
Bit 6	In	T	Bit 6
Bit 7	In	T	Bit 7

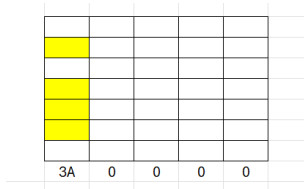
Imagen 3. Configuración del puerto C.

Ports Settings			
PORTA	Port B	Port C	Port D
Data Direction		Pullup/Output Value	
Bit 0	Out	0	Bit 0
Bit 1	Out	0	Bit 1
Bit 2	Out	0	Bit 2
Bit 3	Out	0	Bit 3
Bit 4	Out	0	Bit 4
Bit 5	Out	0	Bit 5
Bit 6	Out	0	Bit 6
Bit 7	In	T	Bit 7

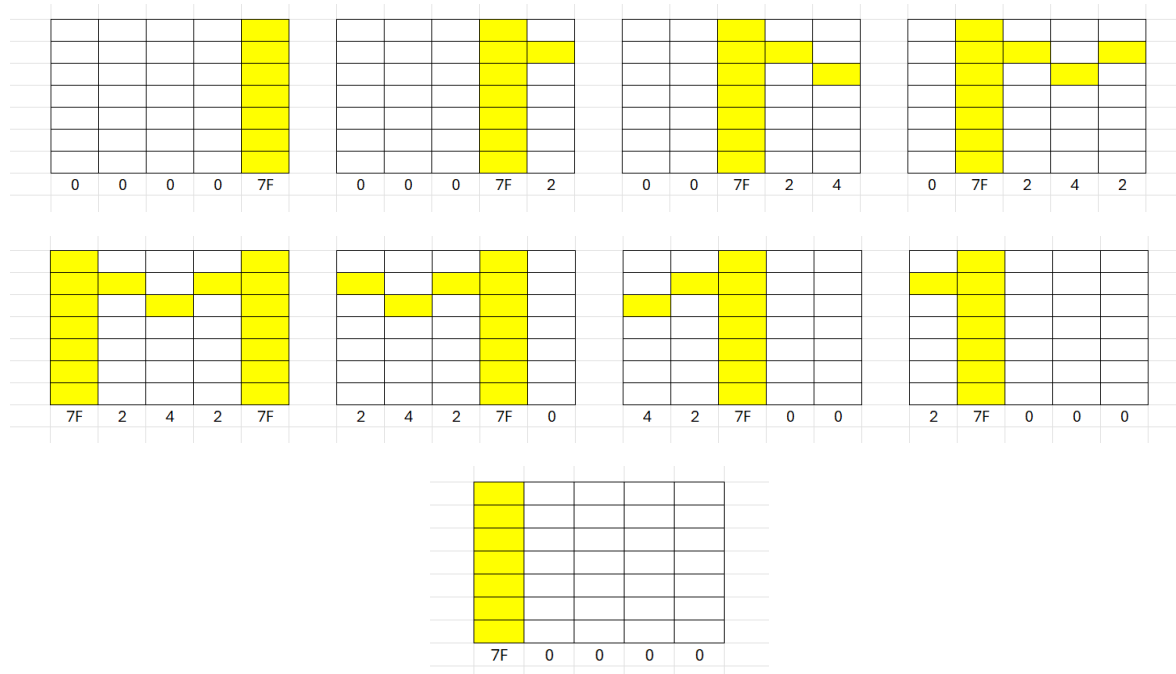
Imagen 4. Configuración del puerto D.

- Pantallas y códigos para la letra G**

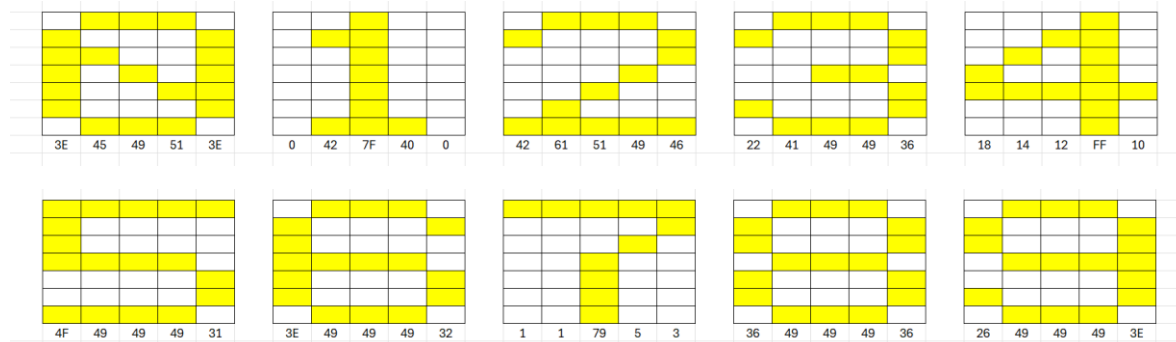




- Pantallas y códigos para la letra M**



- Pantallas y códigos para los números del cero al nueve**



C  digo del circuito de CodeVision:

```
1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4       Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr  ctica 17 'Matriz de led 7x5'
8  Version : 1.1
9  Date    : 12/03/2026
10 Author  : Garc  a Escamilla Bryan Alexos
11 Company :
12 Comments:
13
14
15 Chip type           : ATmega8535
16 Program type        : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model         : Small
19 External RAM size    : 0
20 Data Stack size     : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>
```

```
27 // Declare your global variables here
28 unsigned char modo;
29 char columnas = 0x01;
30 char filas = 0;
31 char indice = 0, numero = 0, letra = 0;
32 char repetir = 0, renglon = 0;
33 char desplazamiento = 0;
34 char buffer_desplazamiento[10] = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9};
35
36 char modo_1[] = {0x7F, 0x7F, 0x7F, 0x7F, 0x7F};
37
38 char modo2_3[7][5] = {
39     {0x01, 0x01, 0x01, 0x01, 0x01},
40     {0x02, 0x02, 0x02, 0x02, 0x02},
41     {0x04, 0x04, 0x04, 0x04, 0x04},
42     {0x08, 0x08, 0x08, 0x08, 0x08},
43     {0x10, 0x10, 0x10, 0x10, 0x10},
44     {0x20, 0x20, 0x20, 0x20, 0x20},
45     {0x40, 0x40, 0x40, 0x40, 0x40}
46 };
47
```

```

48 // Matriz para programar la real
49 char modo_numero[10][5] = {
50     {0x45, 0x49, 0x51, 0x3e, 0x3e}, // 0
51     {0x42, 0x7f, 0x40, 0x00, 0x00}, // 1
52     {0x61, 0x51, 0x49, 0x46, 0x42}, // 2
53     {0x41, 0x49, 0x49, 0x36, 0x22}, // 3
54     {0x14, 0x12, 0x7f, 0x10, 0x18}, // 4
55     {0x49, 0x49, 0x49, 0x31, 0x4f}, // 5
56     {0x49, 0x49, 0x49, 0x32, 0x3e}, // 6
57     {0x01, 0x79, 0x05, 0x03, 0x01}, // 7
58     {0x49, 0x49, 0x49, 0x36, 0x36}, // 8
59     {0x49, 0x49, 0x49, 0x3e, 0x26} // 9
60 };
61
62
63 char modo_nombre[19][5] = {
64     // Letra G real
65     {0x00, 0x00, 0x00, 0x3e, 0x00},
66     {0x00, 0x00, 0x3e, 0x41, 0x00},
67     {0x00, 0x3e, 0x41, 0x49, 0x00},
68     {0x3e, 0x41, 0x49, 0x49, 0x00},
69     {0x41, 0x49, 0x49, 0x3a, 0x3e},
70     {0x49, 0x49, 0x3a, 0x00, 0x41},
71     {0x49, 0x3a, 0x00, 0x00, 0x49},
72     {0x3a, 0x00, 0x00, 0x00, 0x49},
73     {0x00, 0x00, 0x00, 0x00, 0x3a},

```

```

75 // Letra M real
76 {0x00, 0x00, 0x00, 0x7f, 0x00},
77 {0x00, 0x00, 0x7f, 0x02, 0x00},
78 {0x00, 0x7f, 0x02, 0x04, 0x00},
79 {0x7f, 0x02, 0x04, 0x02, 0x00},
80 {0x02, 0x04, 0x02, 0x7f, 0x7f},
81 {0x04, 0x02, 0x7f, 0x00, 0x02},
82 {0x02, 0x7f, 0x00, 0x00, 0x04},
83 {0x7f, 0x00, 0x00, 0x00, 0x02},
84 {0x00, 0x00, 0x00, 0x00, 0x7f}
85 };
86
87 // Funci  n para obtener la columna actual del n  mero en desplazamiento
88 char obtener_columna_desplazamiento(char num_index, char col_index) {
89     // Si el   ndice de columna est   fuera del rango del n  mero, retornar 0 (columna vac  a)
90     if (col_index < 0 || col_index >= 5) {
91         return 0x00;
92     }
93     return modo_numero[num_index][col_index];
94 }

```

```

96 void main(void) {
97     // Declare your local variables here
98
99     // Input/Output Ports initialization
100    // Port A initialization
101    // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
102    DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
103    // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
104    PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
105
106    // Port B initialization
107    // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
108    DDRB=(0<<ddb7) | (0<<ddb6) | (0<<ddb5) | (0<<ddb4) | (0<<ddb3) | (0<<ddb2) | (0<<ddb1) | (0<<ddb0);
109    // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=P Bit0=P
110    PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (1<<PORTB2) | (1<<PORTB1) | (1<<PORTB0);
111
112    // Port C initialization
113    // Function: Bit7=In Bit6=In Bit5=In Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
114    DDRC=(0<<ddc7) | (0<<ddc6) | (0<<ddc5) | (1<<ddc4) | (1<<ddc3) | (1<<ddc2) | (1<<ddc1) | (1<<ddc0);
115    // State: Bit7=T Bit6=T Bit5=T Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
116    PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
117
118    // Port D initialization
119    // Function: Bit7=In Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
120    DDRD=(0<<ddd7) | (1<<ddd6) | (1<<ddd5) | (1<<ddd4) | (1<<ddd3) | (1<<ddd2) | (1<<ddd1) | (1<<ddd0);
121    // State: Bit7=T Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
122    PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (0<<PORTD1) | (0<<PORTD0);

```

```

124    // Timer/Counter 0 initialization
125    // Clock source: System Clock
126    // Clock value: Timer 0 Stopped
127    // Mode: Normal top=0xFF
128    // OC0 output: Disconnected
129    TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
130    TCNT0=0x00;
131    OCR0=0x00;
132
133    // Timer/Counter 1 initialization
134    // Clock source: System Clock
135    // Clock value: Timer1 Stopped
136    // Mode: Normal top=0xFFFF
137    // OC1A output: Disconnected
138    // OC1B output: Disconnected
139    // Noise Canceler: Off
140    // Input Capture on Falling Edge
141    // Timer1 Overflow Interrupt: Off
142    // Input Capture Interrupt: Off
143    // Compare A Match Interrupt: Off
144    // Compare B Match Interrupt: Off
145    TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
146    TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
147    TCNT1H=0x00;
148    TCNT1L=0x00;
149    ICR1H=0x00;
150    ICR1L=0x00;
151    OCR1AH=0x00;
152    OCR1AL=0x00;
153    OCR1BH=0x00;
154    OCR1BL=0x00;

```

```

156 // Timer/Counter 2 initialization
157 // Clock source: System Clock
158 // Clock value: Timer2 Stopped
159 // Mode: Normal top=0xFF
160 // OC2 output: Disconnected
161 ASSR=(0<<AS2);
162 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
163 TCNT2=0x00;
164 OCR2=0x00;
165
166 // Timer(s)/Counter(s) Interrupt(s) initialization
167 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
168
169 // External Interrupt(s) initialization
170 // INT0: Off
171 // INT1: Off
172 // INT2: Off
173 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
174 MCUCSR=(0<<ISC2);
175
176 // USART initialization
177 // USART disabled
178 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);
179
180 // Analog Comparator initialization
181 // Analog Comparator: Off
182 // The Analog Comparator's positive input is
183 // connected to the AIN0 pin
184 // The Analog Comparator's negative input is
185 // connected to the AIN1 pin
186 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
187 SFIOR=(0<<ACME);

```

```

189 // ADC initialization
190 // ADC disabled
191 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
192
193 // SPI initialization
194 // SPI disabled
195 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
196
197 // TWI initialization
198 // TWI disabled
199 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);
200
201 while (1) {
202     // Place your code here
203     modo = PINB & 0x07;
204     switch (modo) {
205         case 0: // LINEAS HORIZONTALES HACIA LA IZQUIERDA
206             filas = ~modo_1[indice];
207             delay_ms(100);
208             break;
209         case 1: // LINEAS VERTICALES HACIA ABAJO
210             filas = ~modo2_3[renglon][indice];
211             repetir++;
212             if (repetir == 5) {
213                 renglon++;
214                 repetir = 0;
215             }
216             if (renglon == 7) renglon = 0;
217             break;

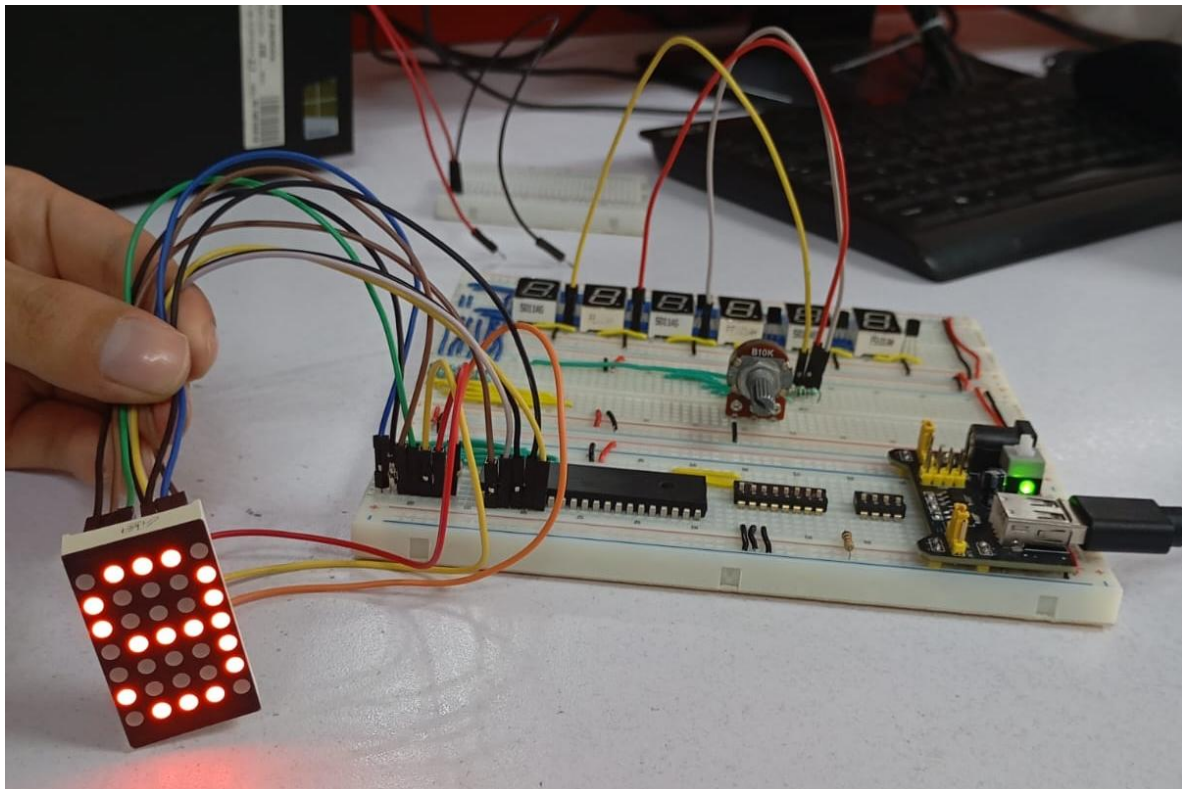
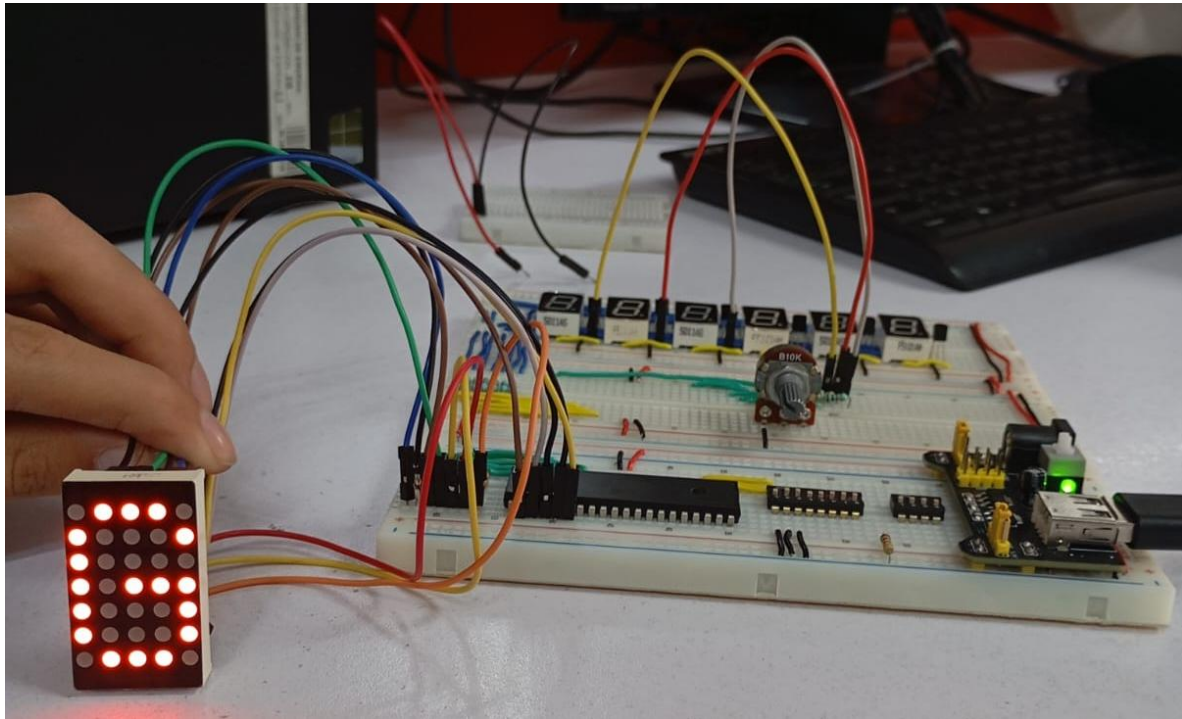
```

```
218 case 2: // PUNTITO HACIA LA IZQUIERDA-ABAJO
219     filas = ~modo2_3[renglon][indice];
220     delay_ms(100);
221     repetir++;
222     if (repetir == 5) {
223         renglon++;
224         repetir = 0;
225     }
226     if (renglon == 7) renglon = 0;
227     break;
228 case 3: // N3MEROS DEL 0 AL 9
229     {
230         char num_actual = buffer_desplazamiento[desplazamiento / 5];
231         char col_actual = indice;
232
233         if (col_actual >= 0 && col_actual < 5) {
234             filas = ~obtener_columna_desplazamiento(num_actual, col_actual);
235         } else {
236             filas = 0xFF; // Columna vac3a
237         }
238
239         repetir++;
240         if (repetir >= 100) {
241             repetir = 0;
242             desplazamiento++;
243             if (desplazamiento >= 50) {
244                 desplazamiento = 0;
245             }
246         }
247     }
248     break;
```

```
249         default:
250             filas = ~modo_nombre[letra][indice];
251             repetir++;
252
253         if (repetir == 60) {
254             repetir = 0;
255             letra++;
256         }
257         if (letra == 19) {
258             letra = 0;
259         }
260     }
261
262     // Necesario para hacer los corrimientos
263     switch (columnas) {
264         case 0x01:
265             columnas = 0x02;
266             break;
267         case 0x02:
268             columnas = 0x04;
269             break;
270         case 0x04:
271             columnas = 0x08;
272             break;
273         case 0x08:
274             columnas = 0x10;
275             break;
276         default:
277             columnas = 0x01;
278     }
```

```
280         indice++;
281         if (indice == 5) {
282             indice = 0;
283         }
284
285         PORTC = columnas;
286         PORTD = filas;
287         delay_ms(2);
288     }
289 }
```

Circuito f3sico



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Al final la práctica sigue el siguiente flujo:

1. Lee el modo de la matriz (actividades)
2. Determina que filas se van a mostrar
3. Activa columna por columna
4. Actualiza filas y columnas en los puertos correspondientes

Dentro de este flujo se encuentran los diferentes modos (actividades):

- a) **MODO 0 (Líneas horizontales):** En este modo se tiene el arreglo `modo_1 = {0x7F, 0x7F, 0x7F, 0x7F, 0x7F}` que representa toda una fila encendida. Obtenemos cada una de las filas con `filas = ~modo_1[indice]`, y como las columnas se activan una por una se genera el efecto de líneas horizontales.
- b) **MODO 1 (Líneas verticales hacia abajo):** En este modo se tiene el arreglo `modo2_3` que contiene las pantallas de como la línea vertical se mueve de arriba hacia abajo. Obtenemos cada una de las filas con `filas = ~modo2_3[renglon][indice]`, recorremos las columnas y cuando se recorran las 5 se vuelve a empezar, y esto genera el efecto de las líneas verticales hacia abajo.
- c) **MODO 2 (Punto en diagonal):** En este modo se repite la lógica del modo 1, solo que con un retardo más grande al obtener las filas lo que permite ver solo un punto por columna en cada fila, manteniendo el efecto de movimiento hacia abajo.
- d) **MODO 3 (Números):** En este modo se selecciona primero el número a mostrar con `char num_actual = buffer_desplazamiento[desplazamiento / 5]`, se obtiene la columna actual con `char col_actual = indice` y mostramos columna por columna del número seleccionado con `filas = ~obtener_columna_desplazamiento(num_actual, col_actual)` y esto se hace 50 veces porque son 5 columnas por 10 números y al llegar a la última columna esta se reinicia y los números vuelven a mostrarse.
- e) **MODO por defecto (Iniciales):** Para este modo se tiene el arreglo `modo_nombre` que contiene las columnas predefinidas de las letras G y M. Se recorre columnas por columnas dando el efecto de desplazamiento.

- **Meléndez Macedonio Rodrigo**

Para esta práctica se desarrolló un circuito donde se destaca el uso de la matriz de LEDs 7x5, la cual configuramos de tal forma que mostrara diferentes modos dependiendo del valor que haya en el switch, en este caso se tenían 5 modos principales, los cuales se mencionan a continuación:

- ✓ **Modo por defecto:** En este modo se mostraron las iniciales de nuestros primeros apellidos (G y M) mediante un desplazamiento hacia la izquierda para mostrar ambos caracteres al mismo tiempo dentro de la matriz.

- ✓ **Modo 0 (Líneas horizontales):** Este modo muestra una línea de LEDs horizontal que empieza desde arriba y se va recorriendo fila por fila para simular un desplazamiento hacia abajo cíclico.
- ✓ **Modo 1 (Líneas verticales):** Este modo muestra como las líneas verticales se mueven hacia abajo de manera continua y al mismo tiempo. Aquí la velocidad del movimiento da la impresión de que todos los LEDs estuvieran encendidos, pero esto no es así, pues el desplazamiento es muy rápido.
- ✓ **Modo 2 (punto en diagonal):** Este modo recrea el desplazamiento de un LED hacia abajo y de manera horizontal recorriendo el LED hacia la derecha poco a poco para dar esa impresión.
- ✓ **Modo 3 (Números):** Este modo recrea un contador de 0 a 9, mostrando cada uno de los números cada cierto tiempo, repitiendo la misma secuencia de manera cíclica.

BIBLIOGRAFÍA

(s.f.). *Multiplexing*. All About Circuits. Recuperado el 17 de marzo de 2026.
<https://www.allaboutcircuits.com/>

(s.f.). *Matriz de LEDs*. Luis Llamas. Recuperado el 17 de marzo de 2026.
<https://www.luisllamas.es/>